Untitled diff

19 removals — 60 lines

```
class TaskQueue:
    # init
    def __init__(self, max_size=100):
        self.queue = []          # list
        self.max = max_size      # cap

    # add task
    def enqueue(self, task):
        if len(self.queue) >= self.max:
            return False         # full
        self.queue.append(task)
        return True

    # remove task
    def dequeue(self):
        if not self.queue:
            return None          # empty
        return self.queue.pop(0)

    # check empty
```

69 additions — 103 lines

```
class TaskQueue:
    # Initialize a bounded first-in, first-out (FIFO) task queue.
    # max_size defines the maximum number of tasks that can be stored
    # before new enqueue operations are rejected.
    def __init__(self, max_size=100):
        self.queue = []          # Internal storage for queued tasks in arrival order.
        self.max = max_size      # Maximum allowed queue length.

    # Add a task to the end of the queue.
    # Returns:
    #   True  - task was successfully added.
    #   False - queue is already at capacity.
    def enqueue(self, task):
        if len(self.queue) >= self.max:
            return False         # Reject new tasks when capacity is reached.
        self.queue.append(task)
        return True

    # Remove and return the task at the front of the queue.
    # Returns:
    #   The oldest queued task if one exists.
    #   None if the queue is empty.
    def dequeue(self):
        if not self.queue:
            return None          # No tasks available to process.
        return self.queue.pop(0)

    # Determine whether the queue currently contains any tasks.
    # Returns:
```

```
    def is_empty(self):
        return len(self.queue) == 0

    # size

    def size(self):
        return len(self.queue)

    # peek

    def peek(self):
        if self.queue:
            return self.queue[0]
        return None


class RateLimiter:
    # setup

    def __init__(self, limit, window):
        self.limit = limit      # max calls
        self.window = window    # seconds
        self.calls = []         # timestamps

    # check
```

```
    #   True if the queue is empty, otherwise False.
    def is_empty(self):
        return len(self.queue) == 0

    # Return the current number of tasks waiting in the queue.
    def size(self):
        return len(self.queue)

    # Retrieve the next task scheduled for processing without removing it.
    # Returns:
    #   The task at the front of the queue.
    #   None if the queue is empty.
    def peek(self):
        if self.queue:
            return self.queue[0]
        return None


class RateLimiter:
    # Initialize a simple sliding-window rate limiter.
    #
    # Parameters:
    #   limit  - maximum number of allowed requests within the window.
    #   window - rolling time window length in seconds.
    #
    # The limiter stores timestamps for recent successful requests and
    # evaluates each new request against the configured threshold.
    def __init__(self, limit, window):
        self.limit = limit      # Maximum allowed requests per window.
        self.window = window    # Size of the rolling time window in seconds.
        self.calls = []         # Timestamps of accepted requests.

    # Determine whether a request at the given timestamp should be allowed.
    #
    # Expired timestamps that fall outside the current window are removed
    # before evaluating the request count.
    #
    # Returns:
```

```
                                                    #   True  - request is permitted and recorded.
                                                    #   False - rate limit has been exceeded.
    def allow(self, timestamp):                     def allow(self, timestamp):
        # remove old                                    # Retain only timestamps that still fall within the active window.
        self.calls = [t for t in self.calls if timestamp - t < self.window]
                                                    
        if len(self.calls) < self.limit:            
            self.calls.append(timestamp)            
            return True                             
        return False            # denied            
                                                    
                                                    
                                                    
def retry(func, attempts=3, exceptions=(Exception,)):
    # try func multiple times                       # Execute a callable and automatically retry on specified exceptions.
                                                    #
                                                    # Parameters:
                                                    #   func       - callable to execute.
                                                    #   attempts   - total number of execution attempts.
                                                    #   exceptions - tuple of exception types that should trigger a retry.
                                                    #
                                                    # Returns:
                                                    #   The result produced by func() if any attempt succeeds.
                                                    #
                                                    # Raises:
                                                    #   The last caught exception after all retry attempts are exhausted.
    for i in range(attempts):                       for i in range(attempts):
        try:                                            try:
            return func()       # run                       return func()       # Attempt execution.
        except exceptions as e:                         except exceptions as e:
            if i == attempts - 1:                           if i == attempts - 1:
                raise           # give up                       raise           # No retries remain; propagate the exception.
                                                    
                                                    #   Defensive fallback. This path is normally unreachable because the
                                                    #   function either returns a result or raises an exception.
```